

Reinforcement Learning

CS 59300: RL1

October 2, 2025

Joseph Campbell
Department of Computer Science

Today's lecture

1. Multi-agent reinforcement learning
2. Challenges in MARL
3. MARL algorithms

Some content inspired by Eugene Vinitsky's guest lecture in Cathy Wu's MIT 6.7950

Multi-agent reinforcement learning

Why do we need multi-agent learning?



Article

Grandmaster level in StarCraft II using multi-agent reinforcement learning

<https://doi.org/10.1038/s41586-019-1724-z>

Received: 30 August 2019

Accepted: 10 October 2019

Published online: 30 October 2019

Oriol Vinyals^{1,2}, Igor Babuschkin^{1,3}, Wojciech M. Czarnecki^{1,2}, Michaël Mathieu^{1,3}, Andrew Dudzik^{1,3}, Junyoung Chung^{1,3}, David H. Choi¹, Richard Power^{1,3}, Timo Ewalds^{1,3}, Petko Georgiev^{1,3}, Junhyuk Oh^{1,3}, Dan Horgan^{1,3}, Manuel Kroiss^{1,3}, Ivo Danihelka^{1,3}, Aja Huang^{1,3}, Laurent Sifre^{1,3}, Trevor Cai^{1,3}, John P. Agapiou^{1,3}, Max Jaderberg, Alexander S. Vezhnevets, Rémi LeBlond, Tobias Pfaffner, Valentín Dalibard, David Budden, Yuhua Yang, James Molloy, Tom N. Paine, Çağlar Gulcehre, Ziyang Wu, Tobias Pfaff, Yuhua Wu, Roman LeBlond, Greg DeSa, Dario Wierstra, Katrine McCreery, Oliver Sherry, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps^{1,3} and David Silver^{1,3,4}

Many real-world applications require artificial agents to compete and coordinate with other agents in complex environments. As a stepping stone to this goal, the domain of StarCraft has emerged as an important challenge for artificial intelligence research, owing to its iconic and enduring status among the most difficult professional esports and its relevance to the real world in terms of its raw complexity and multi-agent challenges. Over the course of a decade and numerous competitions¹⁻³, the strongest agents have simplified important aspects of the game, utilized superhuman capabilities, or employed hand-crafted sub-systems⁴. Despite these advantages, no previous agent has come close to matching the overall skill of top StarCraft players. We chose to address the challenge of StarCraft using general-purpose learning methods that are in principle applicable to other complex domains: a multi-agent reinforcement learning algorithm that uses data from both human and agent games within a diverse league of continually adapting strategies and counter-strategies, each represented by deep neural networks^{5,6}. We evaluated our agent, AlphaStar, in the full game of StarCraft II, through a series of online games against human players. AlphaStar was rated at Grandmaster level for all three StarCraft races and above 99.8% of officially ranked human players.

StarCraft is a real-time strategy game in which players balance high-level economic decisions with individual control of hundreds of units. This domain raises important game-theoretic challenges: it features a vast space of cyclic, non-transitive strategies and counter-strate-

Each action a_i is highly structured: it selects what action type, out of several hundred (for example, move or build worker); who to issue that action to, for any subset of the agent's units; where to target, among locations on the map or units within the camera view; and when to

Why do we need multi-agent learning?



Dota 2 with Large Scale Deep Reinforcement Learning

OpenAI, *

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław "Psycho" Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique Pondé de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, Susan Zhang

December 13, 2019

Abstract

On April 13th, 2019, OpenAI Five became the first AI system to defeat the world champions at an esports game. The game of Dota 2 presents novel challenges for AI systems such as long time horizons, imperfect information, and complex, continuous state-action spaces, all challenges which will become increasingly central to more capable AI systems. OpenAI Five leveraged existing reinforcement learning techniques, scaled to learn from batches of approximately 2 million frames every 2 seconds. We developed a distributed training system and tools for continual training which allowed us to train OpenAI Five for 10 months. By defeating the Dota 2 world champion (Team OG), OpenAI Five demonstrates that self-play reinforcement learning can achieve superhuman performance on a difficult task.

1 Introduction

The long-term goal of artificial intelligence is to solve advanced real-world challenges. Games have served as stepping stones along this path for decades, from Backgammon (1992) to Chess (1997) to Atari (2013)[1-3]. In 2016, AlphaGo defeated the world champion at Go using deep reinforcement learning and Monte Carlo tree search[4]. In recent years, reinforcement learning (RL) models have tackled tasks as varied as robotic manipulation[5], text summarization[6], and video games such as Starcraft[7] and Minecraft[8].

Why do we need multi-agent learning?



Multi-Agent Reinforcement Learning for Resource Allocation in Large-Scale Robotic Warehouse Sortation Centers

Yi Shen¹, Benjamin McClosky², Joseph W. Durham², and Michael M. Zavlanos¹

Abstract—Robotic sortation centers use mobile robots to sort packages by their destinations. The destination-to-sort-location (chute) mapping can significantly impact the volume of packages that can be sorted by the sortation floor. In this work, we propose a multi-agent reinforcement learning method to solve large-scale chute mapping problems with hundreds of agents (the destinations). To address the exponential growth of the state-action space, we decompose the joint action-value function as the sum of local action-value functions associated with the individual agents. To incorporate robot congestion effects on the rates at which packages are sorted, we couple the local action-value functions through the states of destinations mapped to nearby chutes on the sortation floor. We show that our proposed framework can solve large chute mapping problems and outperforms static or reactive policies that are commonly used in practice in robotic sortation facilities.



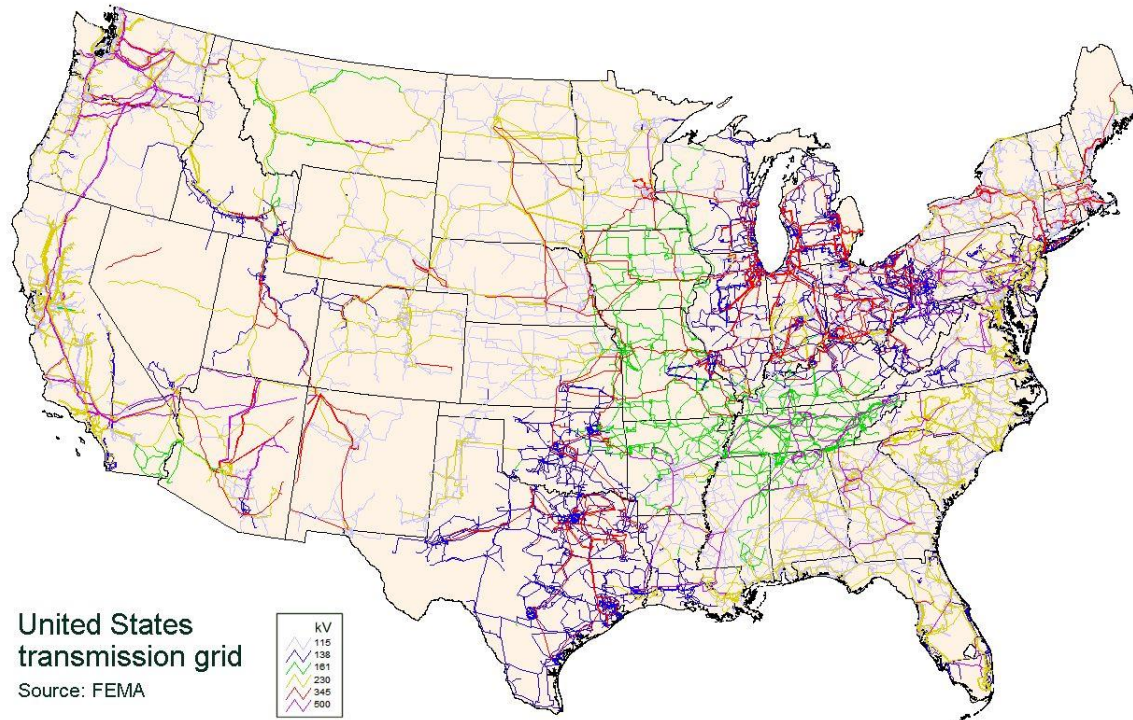
Fig. 1. A sortation floor in an Amazon Robotics sortation hub [3].

I. INTRODUCTION

Modern warehouses often rely on mobile robots to transport and sort packages in their sortation hubs [1]–[4]. For example, in Amazon Robotics sortation hubs [5], packages arrive at induct stations, located on the perimeter of a sortation floor, where they are loaded onto mobile robots and transported based on their destinations to sort locations (eject chutes) arranged in a grid in the interior of the sortation floor; see Fig. 1. In these robotic sortation hubs, the volume of packages (throughput) that can be sorted by the robot fleet is partly determined by the availability of a sufficient number of eject chutes for each destination as well as by how the robots interfere with each other's motion as they navigate from the induct stations to their assigned eject chutes. As a result, the destination-to-chute mapping can significantly impact the throughput that the mobile robot fleet can deliver.

These approaches generally belong to two main categories: queuing networks (QNs) and mathematical programming models (e.g., mixed-integer programs (MIP)). QNs, e.g., open queuing networks [6] or semi-open queuing networks [7], rely on queueing theory to build models of sorting systems and characterize system performance in terms of, e.g., throughput and storage capacities. QNs typically focus on system-level design decisions, also called long-term decisions, such as design of facility layouts [8] or robot zoning strategies [9]; see [10] for a review of robotic sorting methods using QNs. In this paper, we instead focus on operational planning decisions, also called short-term decisions, and in particular, on the destination assignment problem (DAP) [11] assuming that facility layouts, workstation locations, robot path topologies, etc. are given. QNs are not suitable for solving operational-level DAPs; these

Why do we need multi-agent learning?



IEEE TRANSACTIONS ON POWER SYSTEMS, VOL. 35, NO. 6, NOVEMBER 2020 4599

A Multi-Agent Deep Reinforcement Learning Method for Cooperative Load Frequency Control of a Multi-Area Power System

Ziming Yan[✉], Student Member, IEEE, and Yan Xu[✉], Senior Member, IEEE

Abstract—This paper proposes a data-driven cooperative method for load frequency control (LFC) of the multi-area power system based on multi-agent deep reinforcement learning (MA-DRL) in continuous action domain. The proposed method can nonlinearly and adaptively derive the optimal coordinated control strategies for multiple LFC controllers through centralized learning and decentralized implementation. The centralized learning is achieved by MA-DRL based on a global action-value function to quantify overall LFC performance of the power system. To solve the MA-DRL problem, multi-agent deep deterministic policy gradient (DDPG) is derived to adjust control agents' parameters considering the nonlinear generator behaviors. For implementation, each individual controller only needs local information in its control area to deliver optimal control signals. Numerical simulations on a three-area power system and the fully-modeled New-England 39-bus system demonstrate that the proposed method can effectively minimize control errors against stochastic frequency variations caused by load and renewable power fluctuations.

Index Terms—Load frequency control, multi-agent deep reinforcement learning, deep deterministic policy gradient (DDPG), multi-area power systems.

NOMENCLATURE

Variables	
$\Delta f_i, \Delta P_{tie,i}$	Frequency deviation and unscheduled tie-line power flow for i th area.
$\Delta P_{mi}, \Delta P_{gi}$	Generator mechanical output and valve position for i th area.

T_{gi}, T_{ti}	Governor time constant and turbine time constant for i th area.
R_i	Speed drop for i th area.
X_{ij}, T_{ij}	Tie-line reactance and synchronous coefficient between area i and j .
δ_i^0, V_i	Power angle and terminal voltage of equivalent machines of area i .
σ_i	Generation rate constraints for i th area.
Q, a, s	Action-value function, control actions and state information of centralized learning.
θ_i^μ, J	Parameters of i th policy agent and the objective for updating its parameters.
ξ_t	Random number for exploration.
μ	Actor policy to provide control actions (generation commands).
o_i	Observation of i th policy agent.
$\Delta F_i(s), A_i(s)$	Laplace transformation of frequency deviation and control action for i th area.
η	Learning rate to update policy agent.
$\mathbf{W}_i^T, \mathbf{b}_i$	Weighting matrix and bias vector i th deep reinforcement learning neural network.
$\beta_0, \beta_1, \beta_2, \beta_3$	Coefficients to simplify transfer function.
$\Delta t, t$	Time step size and current time indices of fixed time step simulation.
i, j, k	Indices to represent LFC areas and learning iterations.

Other applications

- Autonomous vehicles
 - May be cooperative (street driving) or competitive (racing)
- Human-robot collaboration
- Traffic light management
- Social networks
- Politics/voter modeling
- Economics and market simulations
- Games
- ...

Other applications

Even in centralized problems, multi-agent learning can be *efficient*

Planning and coordinating multiple agents scales super-linearly with the number of agents

- Decomposing the problem into groups of coupled agents is far more efficient
- Centralized vs distributed planners

Recap: Markov Decision Process

We formulate sequential decision-making problems as MDPs

Definition: a tuple consisting of $(\mathcal{S}, \mathcal{A}, R, T, \gamma)$:

- $\mathcal{S}$ is the set of states in our environment
- $\mathcal{A}$ is a set of actions that can be taken by an agent
- R is the reward function. $\mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$
- T is the state transition probability. $\mathcal{S} \times \mathcal{S} \times \mathcal{A} \mapsto [0,1]$
- γ is a discount factor. $\gamma \in [0,1]$

Extension to multi-agent settings

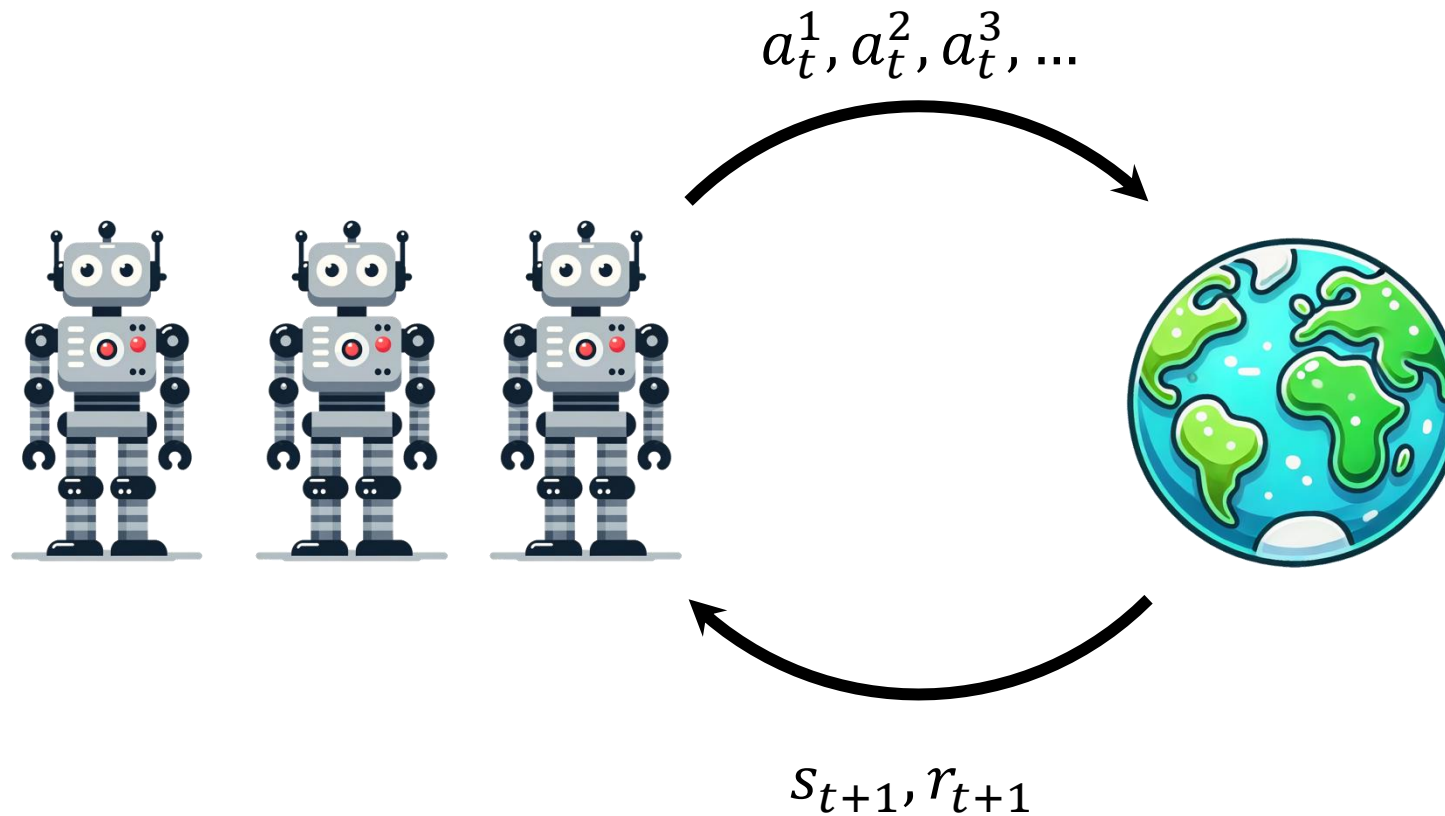
We modify the MDP tuple to account for multiple agents

- May be modeled as either Dec-POMDP or Markov game

Definition: a tuple consisting of $(\mathcal{N}, \mathcal{S}, \mathcal{A}, R, T, \gamma)$:

- $\mathcal{N}$ is the set of agents $\{0 \dots n\}$
- $\mathcal{S}$ is the set of states in our environment
- $\mathcal{A}$ is a set of all agent actions. $\mathcal{A} = \prod_{i \in \mathcal{N}} \mathcal{A}_i$
- R is the reward function. Each agent receives reward $\mathcal{S} \times \mathcal{A}_i \mapsto \mathbb{R}$
 - In a Dec-POMDP all agents receive the same reward for the joint action
- T is the state transition probability. $\mathcal{S} \times \mathcal{S} \times \mathcal{A} \mapsto [0,1]$
- γ is a discount factor. $\gamma \in [0,1]$

Extension to multi-agent settings



Modeling considerations

- Are agents partially observable?
- Do agents receive an individual or team reward?
- Do agents act sequentially or in parallel?
- Do agents cooperate or compete?
- Can agents communicate?

Dec-POMDP

Decentralized Partially Observable Markov Decision Process

- Partially observable
- Shared reward among agents
- Fully cooperative task
- Decentralized agents

Markov game

Extension of MDP to multiple agents

- Partially or fully observable
- Individual rewards (may be cooperative or competitive)
- Cooperative, competitive, or mixed task
- Centralized or decentralized agents

Challenges in MARL

Multiple interacting agents presents problems

What is arguably the biggest challenge in MARL?

Multiple interacting agents presents problems

What is arguably the biggest challenge in MARL?

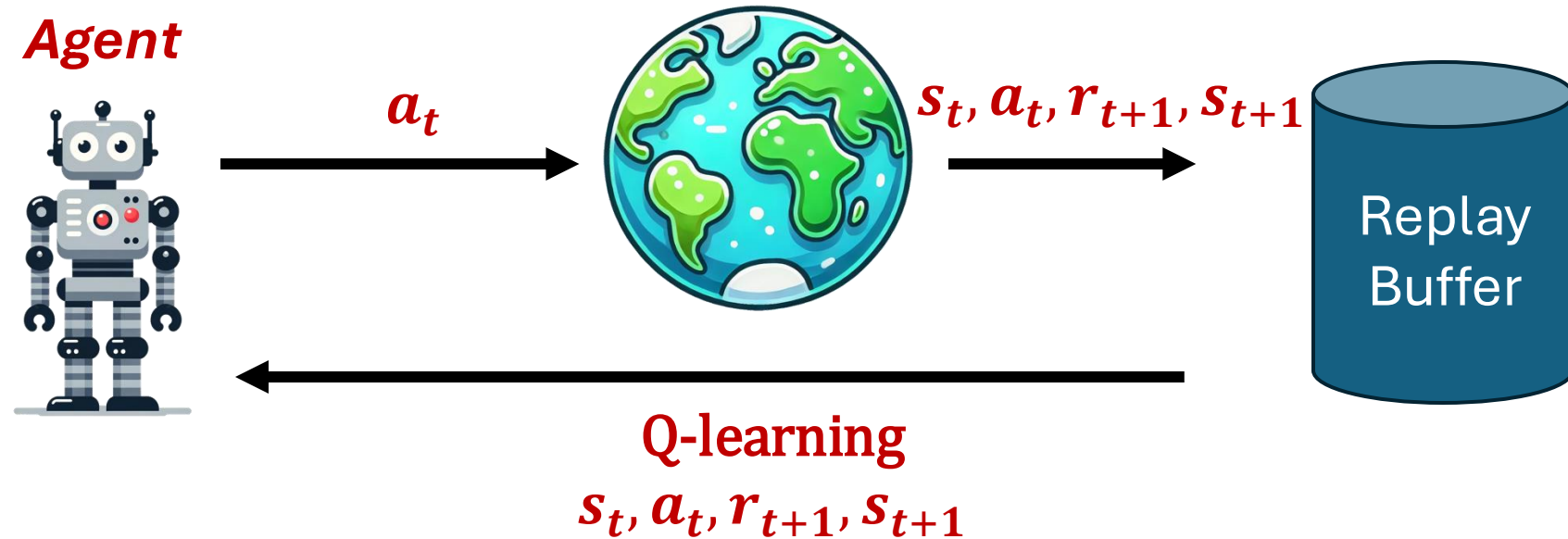
Non-stationarity in the environment dynamics!

Multiple agents are interacting and learning...

- This means their behavior changes over time
- From the perspective of a single agent, this means the dynamics of the environment are changing
- This means all agents must continuously adapt to changing behavior of other agents, creating a feedback cycle

Non-stationarity due to changing policies

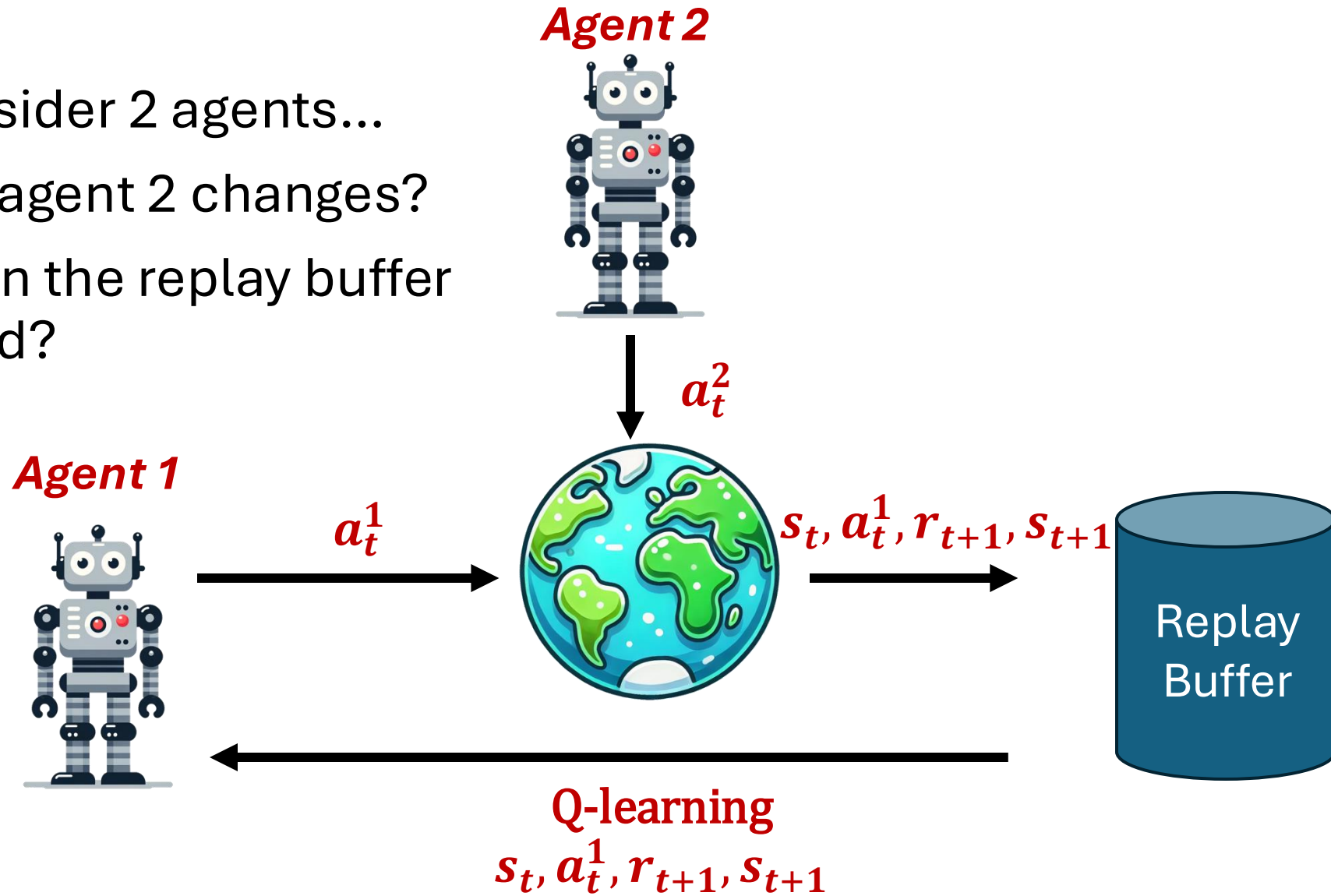
Consider DQN in a single-agent setting...



Non-stationarity due to changing policies

Now consider 2 agents...

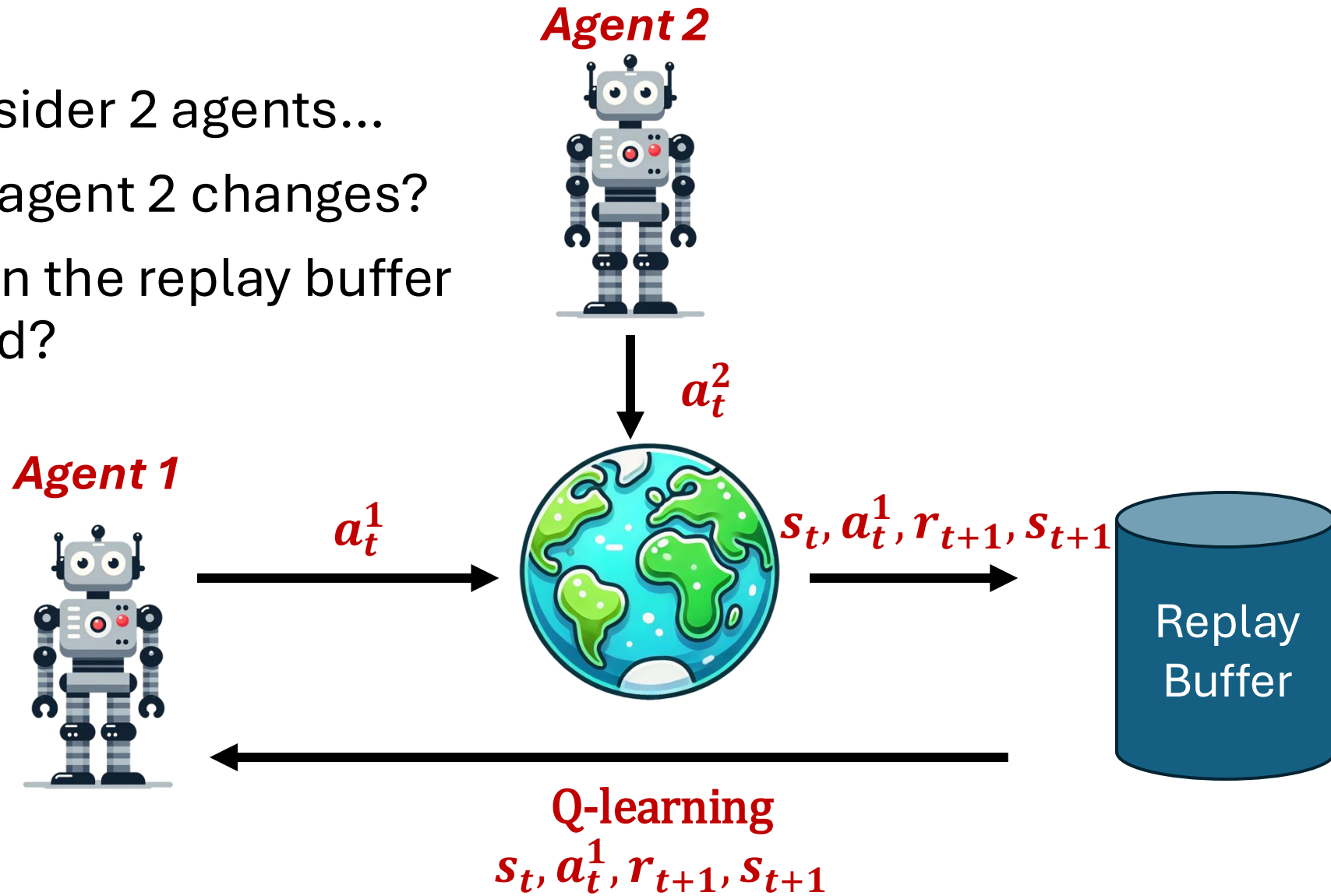
- What if agent 2 changes?
- Is data in the replay buffer still valid?



Non-stationarity due to changing policies

Now consider 2 agents...

- What if agent 2 changes?
- Is data in the replay buffer still valid?
- No!



Non-stationarity due to changing policies

The state transition function depends on the actions of **all agents**

$$p(s_{t+1} | s_t, a_t^1, a_t^2, \dots, a_t^n)$$

If agents are unaware of each others' actions, transition dynamics and expected rewards change if each agent's policy changes!

Stochastic policies in MARL

MDPs have (at least one) optimal, deterministic policy.
Do Dec-POMDPs and Markov games still have one?

No (usually)!

- Dec-POMDP have a stochastic optimal joint policy
 - Requires implicit coordination among agents
 - Partial observability, symmetry breaking, ...
- Markov games have a Nash equilibrium
 - Where no agent has an incentive to deviate from its current strategy
 - In other words, its current strategy is the best response

What is the simplest multi-agent policy?

Predict all agent actions at once:

$$\pi(a_t^1, \dots, a_t^n | s_t)$$

Is this practical?

Hint: consider the action space and credit assignment

Computational tractability of multiple agents

The sample complexity of RL increase with the number of agents

- The (in-)efficiency of RL scales with the dimension of the action space
- If we have n agents and each agent has $|\mathcal{A}|$ actions, then we have $|\mathcal{A}|^n$ actions
- Exponential! This is the **curse of dimensionality**

Credit assignment in multi-agent settings

Recall the **credit assignment problem**

- **Temporal credit:** which actions led to obtaining **rewards**?

In multi-agent settings this has an additional dimension

- **Agent credit:** which agents led to obtaining the **team reward**?

Policies operating in the joint-action space struggle with this

MARL algorithms

How do we overcome non-stationarity?

Idea: use privileged information during training

Agents struggle with disentangling the effect of their own actions in the environment from the (changing) actions of other agents

Solution: provide additional information on other agents during training

Lowe et al, Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments, 2017

Centralized training, decentralized execution

Use a **centralized critic** during training

$$Q_i(\underbrace{s_1, \dots, s_n}_{\text{states of all agents}}, \underbrace{a_1, \dots, a_n}_{\text{actions of all agents}})$$

Include **states of *all agents***

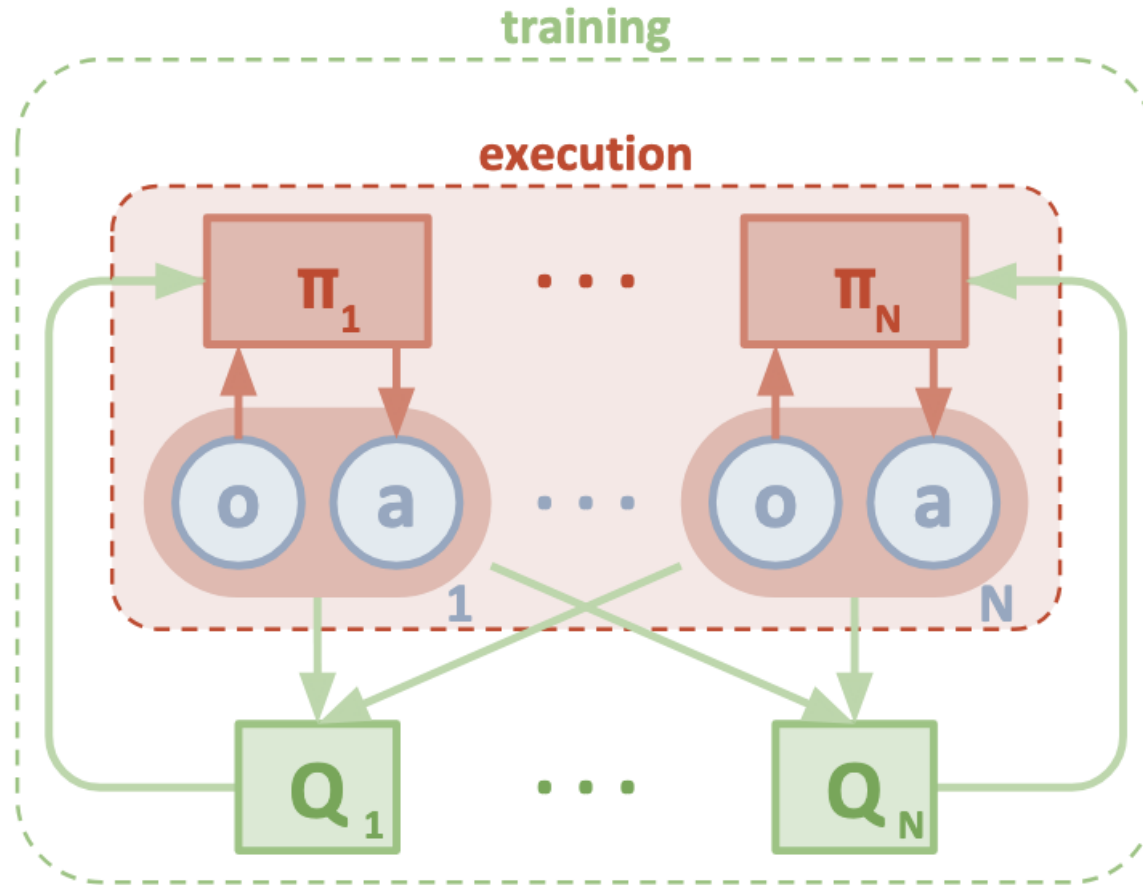
Include **actions of *all agents***

This makes data in the replay buffer *stationary*!

Actor continues to use limited information so execution is **decentralized**

$$\pi_i(a_i | s_i)$$

Centralized training, decentralized execution



Why does this work?

If we know the actions of all agents...

$$\begin{aligned} p(s_{t+1}|s_t, a_t^1, \dots, a_t^n) &= p(s_{t+1}|s_t, a_t^1, \dots, a_t^n, \pi_t^1, \dots, \pi_t^n) \\ &= p(s_{t+1}|s_t, a_t^1, \dots, a_t^n, \pi_t'^1, \dots, \pi_t'^n) \end{aligned}$$

Then the environment is stationary for any policy $\pi \neq \pi'$

Even though the policy uses *local* states, it now has a **stationary critic signal for TD targets** which stabilizes training

Agents can approximate other policies

This means each agent needs privileged information of all actions

$$Q_i(s_1, \dots, s_n, a_1, \dots, a_n)$$

We can avoid this by approximating the policies of other agents

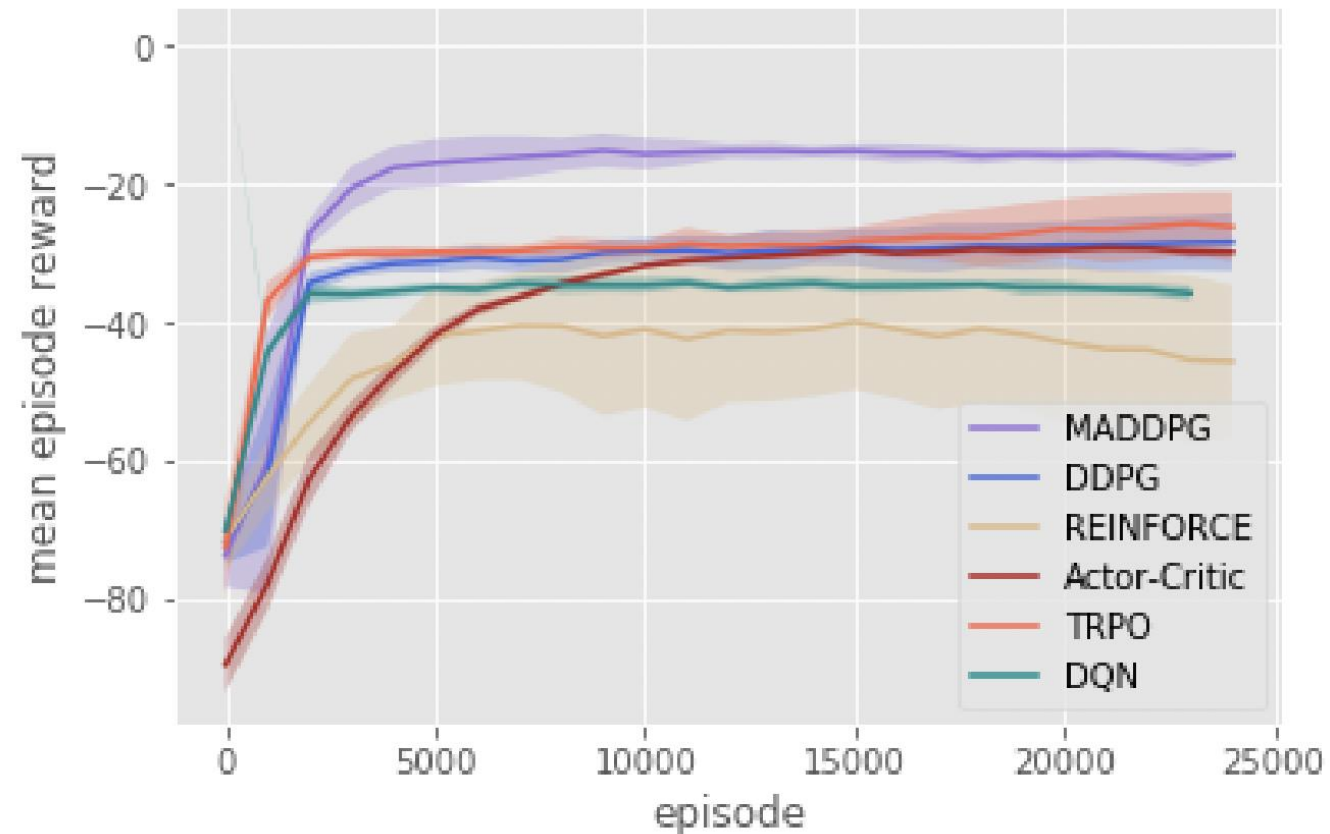
$$\mathcal{L}(\phi_i^j) = -\mathbb{E}_{o_j, a_j} [\log \hat{\pi}_i^j(a_j | o_j) + \lambda H(\hat{\pi}_i^j)]$$

Approximated policy for agent j by agent i

Entropy regularizer

Multi-agent DDPG

Strong performance when applied to DDPG



Further improving efficiency

Decentralized execution addresses the curse of dimensionality

However, for n agents this requires learning n policies

- Very parameter inefficient, as this requires $n * |\theta|$ parameters

Can we avoid this overhead?

Further improving efficiency

Simplest strategy: parameter sharing!

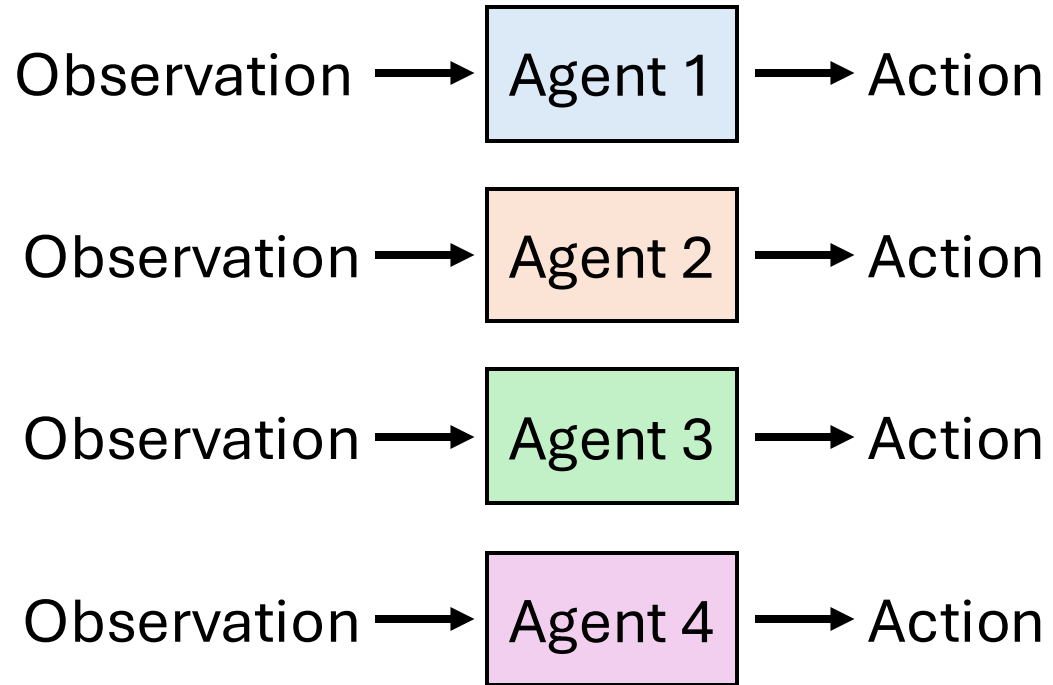
Idea: each agent uses the same underlying network.

- Reduces the number of parameters to learn, instead of $n \times |\theta|$ only need to learn $|\theta|$
- Since all agents use the same policy, makes it easier to generalize to others' behavior

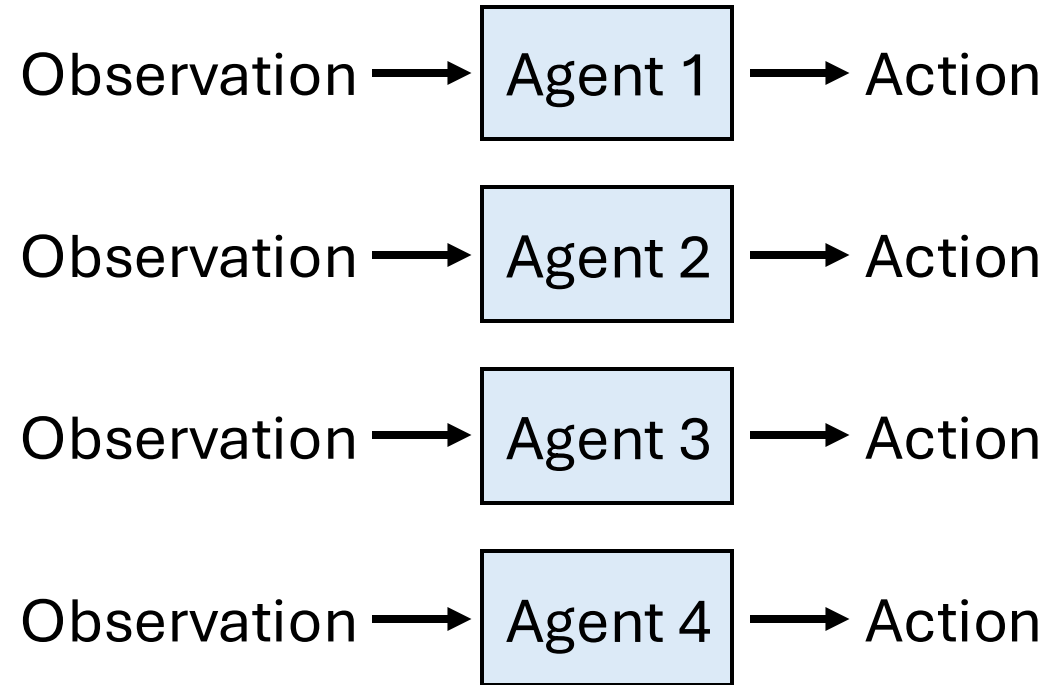
Gupta et al, Cooperative Multi-Agent Control Using Deep Reinforcement Learning, 2017.

Parameter sharing

No sharing



Sharing



Parameter sharing

Pros:

- More computationally efficient
- Can effectively use (on-policy) data from all agents
- Enables implicit coordination as all agents behave similarly

Cons:

- All agents behave similarly (suboptimal for heterogeneous agents)
- Can make credit assignment more difficult (which agent caused it)

The credit assignment problem in MARL

Multiple agents must work together to optimize a **team reward**

- In cooperative tasks agents commonly receive shared reward

Non-stationarity and challenges in credit assignment make purely decentralized learning difficult

- Spurious reward signals from other agents' (unobserved) behavior makes it hard to properly attribute credit

Two settings: **implicit** and **explicit** coordination

Implicit coordination

How can agents learn to coordinate without explicitly doing so?

This means no communication!

- I watch what you do and you watch what I do
- Together we work together to accomplish our goal

Value decomposition networks

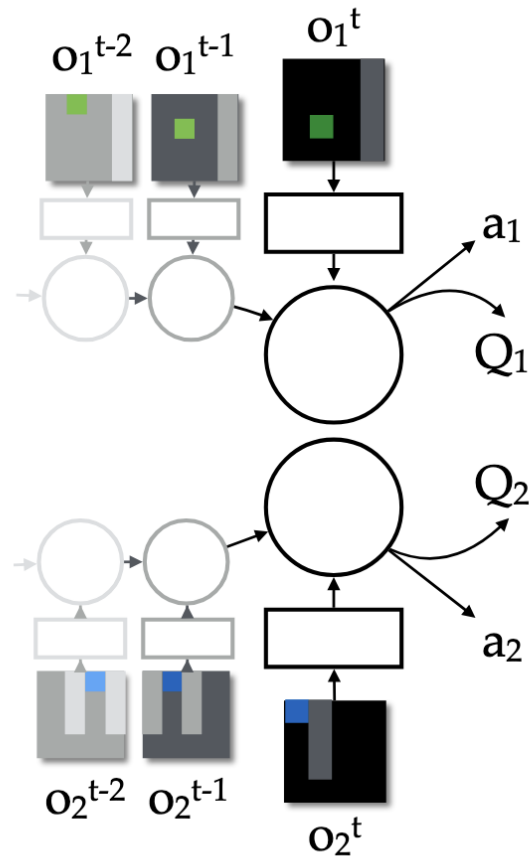
Idea: decompose the team reward into individual contributions

Value decomposition networks:

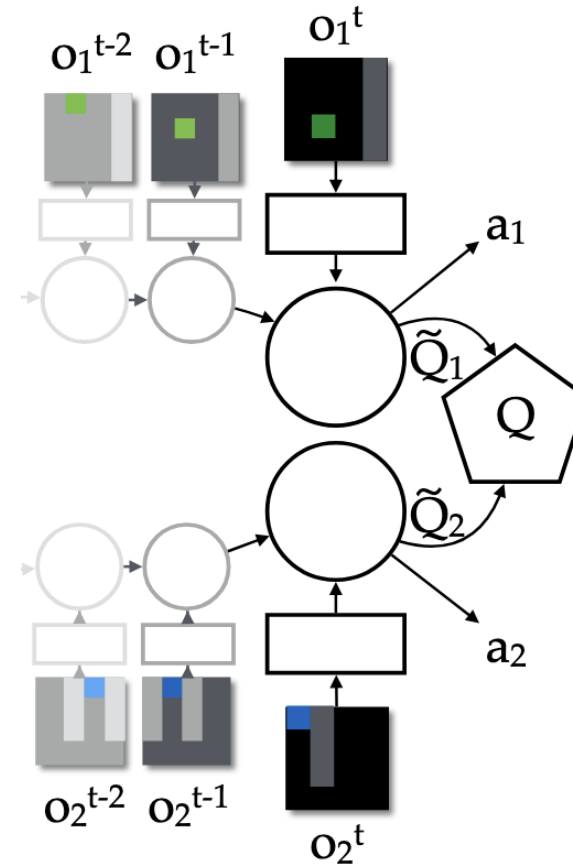
$$Q_{\text{team}}(s, a_1, \dots, a_n) = \sum_i Q_i(s, a_i)$$

Sunchag et al, Value-Decomposition Networks for Cooperative Multi-Agent Learning, 2017.

Value decomposition networks



No decomposition



Decomposition

Why does this work?

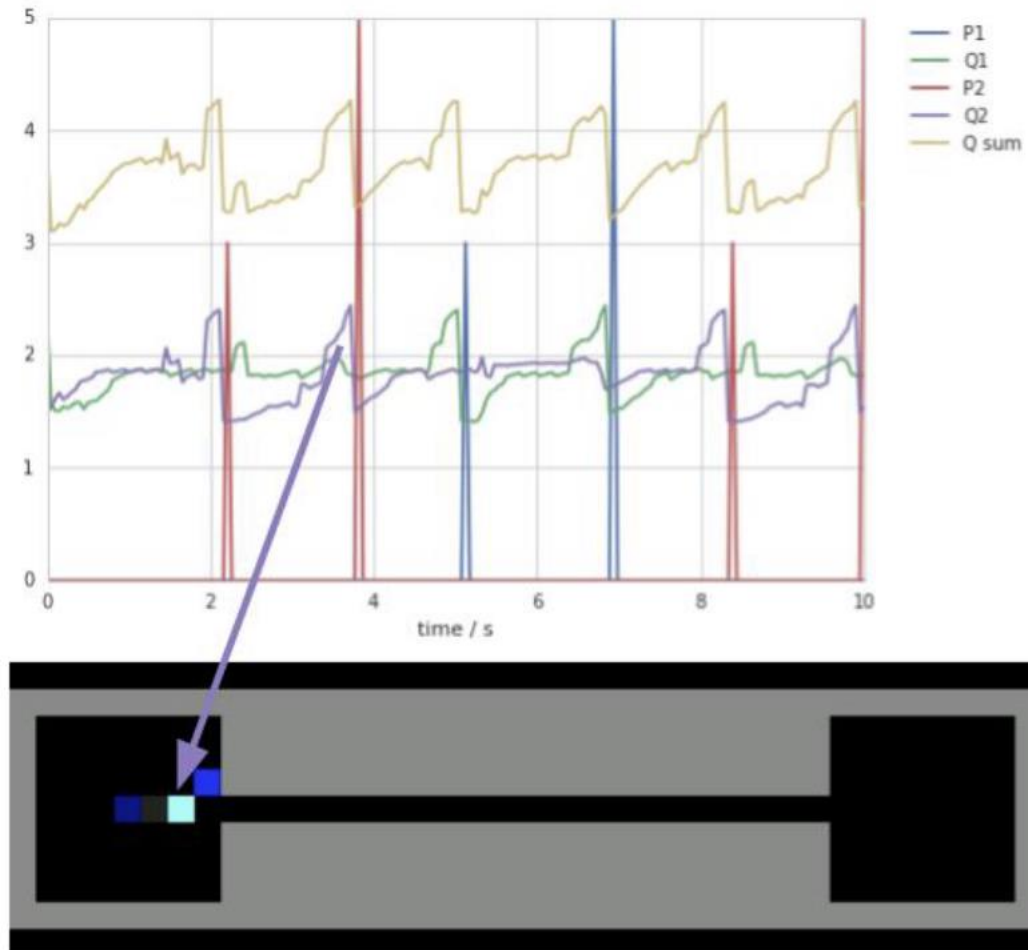
Avoids the “lazy agent” problem

- One agent learns a successful policy, and the other agent is encouraged to do nothing so it doesn't interfere

By using only **local information**, agents learn “how good is my action given what I can see”

Team collectively optimizes total team reward

Value decomposition networks



QMIX

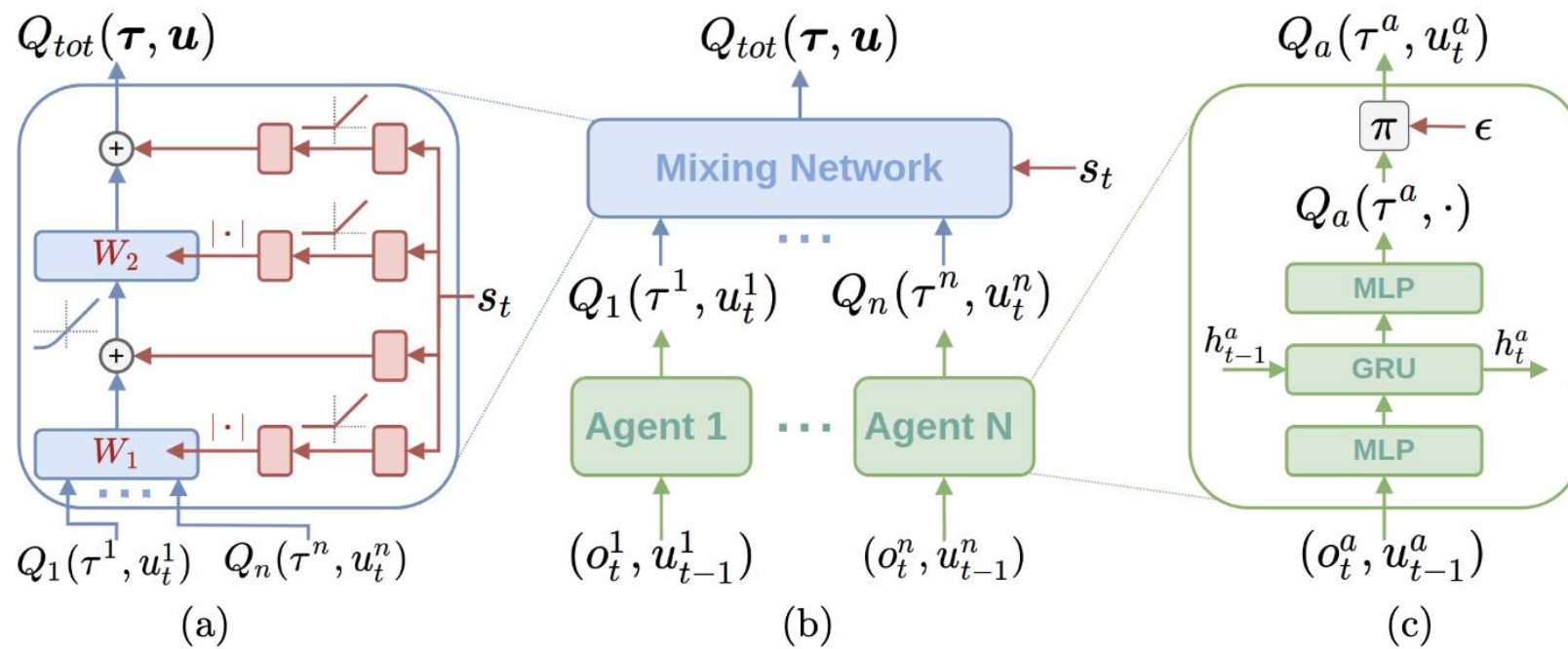
QMIX is a generalization of VDN

- VDN: team value is linear combination of individual values
- QMIX: team value is nonlinear combination of individual values

$$Q_{\text{team}}(s, a_1, \dots, a_n) = f_{\theta}(Q_1(s, a_1), \dots, Q_n(s, a_n))$$

Rashid et al, Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning, 2020.

QMIX



Fun aside: theory of mind

Agents infer other agents' beliefs

- *“I think Agent B doesn’t know that I have the key to the door”*

Cooperative:

- Behave predictably to maximize other agents' belief accuracy
- *“I will move directly to the door”*

Competitive:

- Behave unpredictably to minimize accuracy
- *“I will move to the door in a roundabout way”*

Explicit coordination

Rather than relying on agents learning to implicitly coordinate, what if they are allowed to communicate?

Communication can be either:

- Pre-defined tokens (tell agents what to communicate)
- Learned tokens (agents learn what to communicate)

RIAL/DIAL

Agents have access to limited-bandwidth communication channel

- Learn not just *when* to communicate but *what*

RIAL: agents communicate discrete tokens

- Feedback only from environment

DIAL: agents share network gradients

- Feedback directly from agents

Learning to Communicate with Deep Multi-Agent Reinforcement Learning

Jakob N. Foerster^{1,†}
jakob.foerster@cs.ox.ac.uk

Yannis M. Assael^{1,†}
yannis.assael@cs.ox.ac.uk

Nando de Freitas^{1,2,3}
nandodefreitas@google.com

Shimon Whiteson¹
shimon.whiteson@cs.ox.ac.uk

¹University of Oxford, United Kingdom

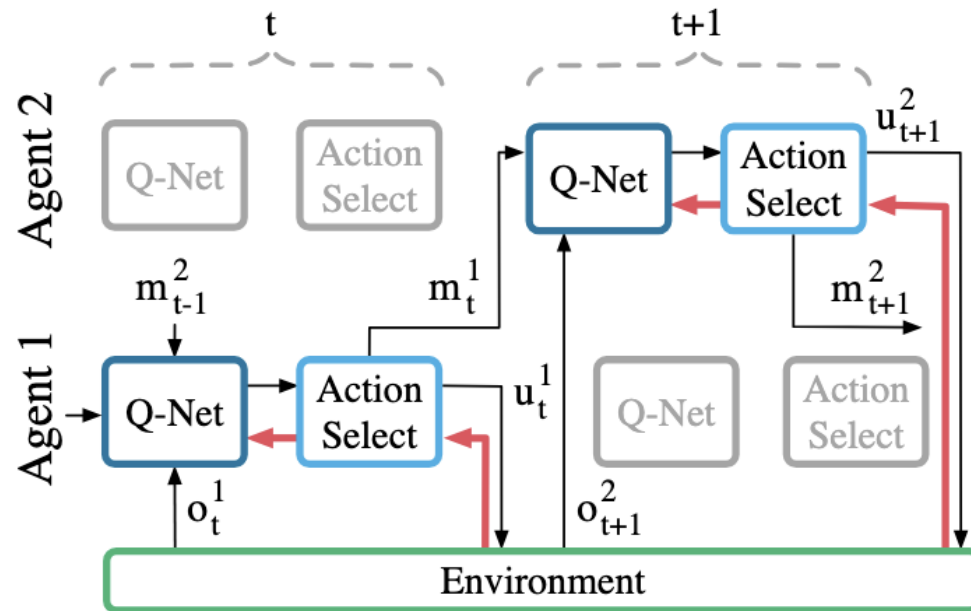
²Canadian Institute for Advanced Research, CIFAR NCAP Program

³Google DeepMind

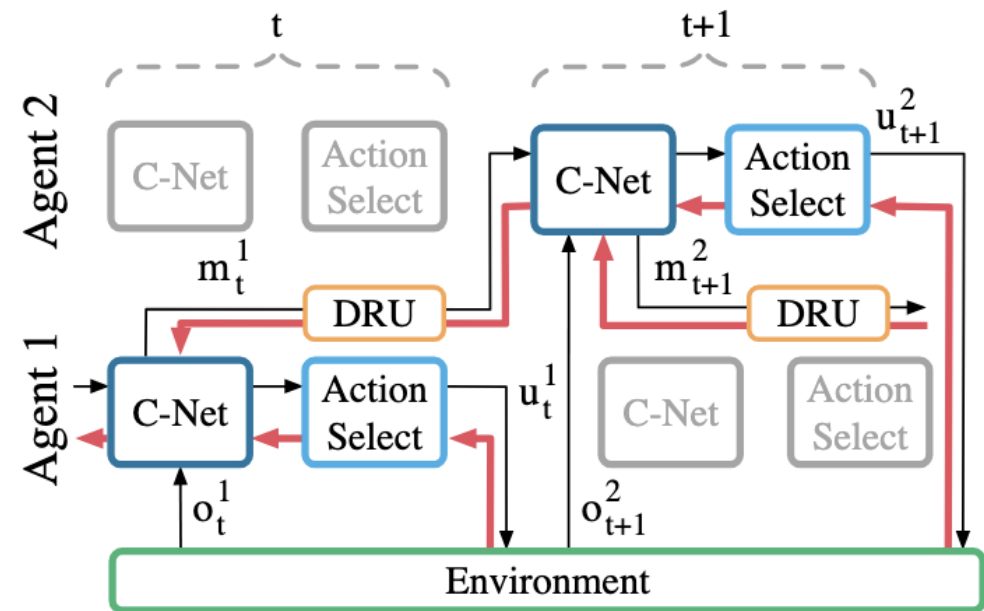
Abstract

We consider the problem of multiple agents sensing and acting in environments with the goal of maximising their shared utility. In these environments, agents must learn communication protocols in order to share information that is needed to solve the tasks. By embracing deep neural networks, we are able to demonstrate end-to-end learning of protocols in complex environments inspired by communication riddles and multi-agent computer vision problems with partial observability. We propose two approaches for learning in these domains: Reinforced Inter-Agent Learning (RIAL) and Differentiable Inter-Agent Learning (DIAL). The former uses deep Q-learning, while the latter exploits the fact that, during learning, agents can backpropagate error derivatives through (noisy) communication channels. Hence, this approach uses centralised learning but decentralised execution. Our experiments introduce new environments for studying the learning of communication protocols and present a set of engineering innovations that are essential for success in these domains.

RIAL/DIAL



(a) RIAL - RL based communication



(b) DIAL - Differentiable communication

Social influence

Use **social influence** to shape communication protocols

- The effect that one agent's actions have on the behavior of other agents

Add additional reward to incentivize agent to communicate in a way that maximally changes other agents' behavior

Social Influence as Intrinsic Motivation for Multi-Agent Deep Reinforcement Learning

Natasha Jaques^{1,2} Angeliki Lazaridou² Edward Hughes² Caglar Gulcehre² Pedro A. Ortega² DJ Strouse³
Joel Z. Leibo² Nando de Freitas²

Abstract

We propose a unified mechanism for achieving coordination and communication in Multi-Agent Reinforcement Learning (MARL), through rewarding agents for having causal influence over other agents' actions. Causal influence is assessed using counterfactual reasoning. At each timestep, an agent simulates alternate actions that it could have taken, and computes their effect on the behavior of other agents. Actions that lead to bigger changes in other agents' behavior are considered influential and are rewarded. We show that this is equivalent to rewarding agents for having high mutual information between their actions. Empirical results demonstrate that influence leads to enhanced coordination and communication in challenging social dilemma environments, dramatically increasing the learning curves of the deep RL agents, and leading to more meaningful learned communication protocols. The influence rewards for all agents can be computed in a decentralized way by enabling agents to learn a model of other agents using deep neural networks. In contrast, key previous works on emergent communication in the MARL setting were unable to learn diverse policies in a decentralized manner and had to resort to centralized training. Consequently, the influence reward opens up a window of new opportunities for research in this area.

across a variety of tasks and environments, sometimes in the absence of environmental reward ([Singh et al., 2004](#)). Previous approaches to intrinsic motivation often focus on curiosity (e.g. [Pathak et al. \(2017\)](#); [Schmidhuber \(2010\)](#)), or empowerment (e.g. [Klyubin et al. \(2005\)](#); [Mohamed & Rezende \(2015\)](#)). Here, we consider the problem of deriving intrinsic social motivation from other agents in multi-agent RL (MARL). Social learning is incredibly important for humans, and has been linked to our ability to achieve unprecedented progress and coordination on a massive scale ([Henrich, 2015](#); [Harari, 2014](#); [Laland, 2017](#); [van Schaik & Burkart, 2011](#); [Herrmann et al., 2007](#)). While some previous work has investigated intrinsic social motivation for RL (e.g. [Sequeira et al. \(2011\)](#); [Hughes et al. \(2018\)](#); [Peysakhovich & Lerer \(2018\)](#)), these approaches rely on hand-crafted rewards specific to the environment, or allowing agents to view the rewards obtained by other agents. Such assumptions make it impossible to achieve independent training of MARL agents across multiple environments.

Achieving coordination among agents in MARL still remains a difficult problem. Prior work in this domain (e.g., [Foerster et al. \(2017; 2016\)](#)), often resorts to centralized training to ensure that agents learn to coordinate. While communication among agents could help with coordination, training emergent communication protocols also remains a challenging problem; recent empirical results underscore the difficulty of learning meaningful emergent communication protocols, even when relying on centralized training (e.g., [Lazaridou et al. \(2018\)](#); [Cao et al. \(2018\)](#); [Foerster et al. \(2016\)](#)).

Training with stochastic policies

Self-play was famously used to train AlphaGo

- Algorithm starts with a random policy
- At each iteration, find a new policy that beats the old one

Many variants:

- E.g., occasionally play against previous models to avoid regression

Dealing with stochastic policies

This works because Go is a turn-based, perfect information game

Go is “close” to transitive

- If A beats B, and B beats C, then A beats C
- Stronger strategies tends to beat weaker strategies, broadly speaking

In more complicated, dynamic games with a more diverse strategy set this no longer works!

- Starcraft 2: partially observable, multi-unit coordination, continuous actions, heterogeneous units, ...

Simple example: rock, paper, scissors

Suppose we use self-play to train on rock, paper, scissors.

Policy 0 = 30% rock, 30% paper, 40% scissors.

Policy 1 = ?

Simple example: rock, paper, scissors

Suppose we use self-play to train on rock, paper, scissors.

Policy 0 = 30% rock, 30% paper, 40% scissors.

Policy 1 = 100% rock.

Policy 2 = ?

Simple example: rock, paper, scissors

Suppose we use self-play to train on rock, paper, scissors.

Policy 0 = 30% rock, 30% paper, 40% scissors.

Policy 1 = 100% rock.

Policy 2 = 100% paper.

Policy 3 = ?

Simple example: rock, paper, scissors

Suppose we use self-play to train on rock, paper, scissors.

Policy 0 = 30% rock, 30% paper, 40% scissors.

Policy 1 = 100% rock.

Policy 2 = 100% paper.

Policy 3 = 100% scissors.

This is an example of **cyclical dynamics**.

- No single, optimal strategy

Before: self-play. Now: league-play

Instead of the agent playing against itself...

1. Keep pool of policies for each player
2. Compute distribution over policies for each player
 - A meta-strategy
3. One agent learns best response to the other's meta-strategy
4. Add best response to the pool of policies

League-play used in AlphaStar

Starcraft 2 exhibits cyclical dynamics

- Three races: Zerg, Terran, Protoss
- Certain units counter other units, much like RPS

AlphaStar uses three policy pools:

- Main agents: 35% self-play, 50% play against old agents, 15% exploiters
- League exploiters: 100% play against old agents
- Main exploiters: 100% play against main agents



However...simpler is sometimes better

PPO + centralized critic + parameter sharing = pretty good

The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games

Chao Yu^{1‡}, Akash Velu^{2‡*}, Eugene Vinyals^{2b}, Jiaxuan Gao¹,
Yu Wang^{1b}, Alexandre Bayen², Yi Wu^{13b}

¹ Tsinghua University ² University of California, Berkeley ³ Shanghai Qi Zhi Institute
[‡]zoeyuchao@gmail.com, ^{*}akashvelu@berkeley.edu

Abstract

Proximal Policy Optimization (PPO) is a ubiquitous on-policy reinforcement learning algorithm but is significantly less utilized than off-policy learning algorithms in multi-agent settings. This is often due to the belief that PPO is significantly less sample efficient than off-policy methods in multi-agent systems. In this work, we carefully study the performance of PPO in cooperative multi-agent settings. We show that PPO-based multi-agent algorithms achieve surprisingly strong performance in four popular multi-agent testbeds: the particle-world environments, the StarCraft multi-agent challenge, Google Research Football, and the Hanabi challenge, with minimal hyperparameter tuning and without any domain-specific algorithmic modifications or architectures. Importantly, compared to competitive off-policy methods, PPO often achieves competitive or superior results in both final returns and sample efficiency. Finally, through ablation studies, we analyze implementation and hyperparameter factors that are critical to PPO's empirical performance, and give concrete practical suggestions regarding these factors. Our results show that when using these practices, simple PPO-based methods can be a strong baseline in cooperative multi-agent reinforcement learning. Source code is released at <https://github.com/marlbenchmark/on-policy>.

1 Introduction

Recent advances in reinforcement learning (RL) and multi-agent reinforcement learning (MARL) have led to a great deal of progress in creating artificial agents which can cooperate to solve tasks: DeepMind's AlphaStar surpassed professional-level performance in the StarCraft II [35], OpenAI

Take-aways

MARL introduces many new challenges:

- Non-stationarity
- Stochasticity
- Computational tractability

Can overcome with a variety of methods: CTDE, parameter sharing, implicit/explicit communication